

Assignment - 3

Module 15 – Advanced Python Programming

➤ **Printing on Screen**

1. Introduction to the print() function in Python.

⇒ It is used to display output on the screen.

It is most commonly used functions, especially for beginners.

Syntax :- print(object)

Example :- print("Hello")

2. Formatting outputs using f-strings and format().

⇒ Formatted output helps you insert variable and values into string neatly and clearly.

➤ **F-string :-**

Syntax :-

F"text {variable}"

Example :-

name = "Yash"

email = "yash@gmail.com"

age = 25

print(f"My Name is {name} and email is {email} and i am {age} years old!...")

➤ **Format () :-**

Example :-

name = "Yash"

email = "yash@gmail.com"

age = 25

txt1 = print("my name is {0} and also my age is {2}".format(name,email,age))

print(txt1)

3. Using the input() function to read user input from the keyboard.

⇒ Input function is used to take a input from the user through keyboard.

Syntax :- input("Message")

Example :-

```
Name = input("Enter Name = ")  
print("Hello" + Name)
```

4. Converting user input into different data types (e.g., int, float, etc.).

⇒ Input function always return data as a string.

To use the input as numbers or other data types, we converted using type conversion like int(), float() and bool()

Example :-

```
Age = int(input("Enter Age = ")  
Print("Age = ",Age)
```

5. Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').

⇒ Open() function lets you open files in different modes depending on what you want to do. (r,w,a,r+,w+,a+)

Syntax :-

```
File = open('filename.txt', "mode")
```

Explanation :-

'r' :-

Open file for reading only.
Error if file doesn't exist.
Cursor starts at begininig

Example :-

```
File = open('test.txt','r')  
Content = file.read()  
Print(Content)
```

File.close()

'w' :-

Open file for writing

Create file doesn't exist

Delete existing content before writing.

Example :-

```
File = open('test.txt','w')
```

```
File.write("Hello World")
```

```
File.close()
```

'a' :-

Add new data at the end

Create file if it doesn't exist.

Existing content remain

```
File = open('test.txt','a')
```

```
File.write("\n new line added")
```

```
File.close()
```

'r+' :-

Allows both reading and writing

File must be exist

Does not erase existing content

Cursor starts at beginning.

Example :-

```
file = open("data.txt", "r+")
```

```
print(file.read())
```

```
file.write("\nExtra text")
```

```
file.close()
```

'W+' :-

Read + write

Create file if missing

Erases existing content

Example :-

```
File = open("data.txt","w+")
```

```
File.write("New Content")
```

```
File.seek(0)
```

```
Print(file.read())
```

```
File.close()
```

6. Using the open() function to create and access files.

⇒ Open() function in python is used to :

- Create files
- Open Existing files
- Read data from files
- Write data to files

Syntax :- `file_object = open("file_name","mode")`

There are basically common modes

R :- Read

W :- Write

A :- Append

R+ :- Read / write

W+ :- Write / Read

X :- create new file

7. Closing files using close().

⇒ after working with the file, it is important to close it using the close() method.

closing a file :

Free system resources

Saves changes properly

Prevents data corruption

Syntax :-

```
File.close()
```

Example :-

```
File = open("test.txt","w")
```

```
File.write("Hello World")
```

```
File.close()
```

8. Reading from a file using read(), readline(), readlines().

⇒ There are below explanations :-

➤ **Read()** :- reads the entire file content as a single string.

Example :-

```
File = open('demo.txt')
```

```
Content = file.read()
```

```
Print(content)
```

```
File.close()
```

➤ **Readline()** :- reads only one line at a time

Example :-

```
File = open('demo.txt','r')
```

```
Line1 = file.readline()
```

```
Line2 = file.readline()
```

```
Print(Line1)
```

```
Print(Line2)
```

File.close()

- **Readlines** :- read all lines and store in the list

Example :-

```
File = open('demo.txt','r')
```

```
Line1 = file.readlines()
```

```
Print(Line1)
```

```
File.close()
```

9. Writing to a file using write() and writelines().

⇒ Below the explanation of two methods :-

- **Write()** :-

Write a single string to the file

Return the number of characters written.

Syntax :- file.write(string)

Example :-

```
F = open('demo.txt','w')
```

```
f.write("Hello Yash")
```

```
f.close()
```

- **Writelines()** :-

Write multiple strings from a list or sequence.

Does not automatically add new line characters (\n)

Syntax :-

```
File.writelines(list_of_strings)
```

Example :-

```
File = open('demo.txt', 'w')
```

```
Lines = ["Python\n", "Java\n", "HTML"]
```

```
File.writelines(Lines)
```

```
File.close()
```

10. Introduction to exceptions and how to handle them using try, except, and finally.

⇒ An exception is an error that occurs during program execution and disrupts the normal flow of the program.

Why Exception occurs :-

- Division by zero
- File Not Found
- Invalid Input
- Using undefined variable

✓ **Try Block :-**

- Code they may cause an error is written inside try.

✓ **Except :-**

- Handles the error if it is occurs
- Prevents program from crashing.

✓ **Finally :-**

- Always Executable.
- Used for clean up actions(like closing files)

Syntax :-

Try:

```
#risky code
```

Except Exception as e:

```
#handle errors
```

```
Finally:
```

```
#Always Executable
```

Example :-

```
Try:
```

```
A = 10
```

```
B = 10/0
```

```
Print(B)
```

```
Except Exception as e:
```

```
Print(e)
```

```
Finally:
```

```
Print("Always Executable")
```

11. Understanding multiple exceptions and custom exceptions.

⇒ There are multiple exceptions and custom exceptions below:-

➤ **Multiple Exceptions :-**

A program may produce different types of errors.

A python allows handling them using multiple except blocks.

Example :-

```
try:
```

```
a = int(input("Enter a number: "))
```

```
b = int(input("Enter another number: "))
```

```
result = a / b
```

```
print("Result:", result)
```



```
except ZeroDivisionError:
```

```
    print("Cannot divide by zero!")
```

```
except ValueError:
```

```
    print("Invalid input! Enter numbers only.")
```

➤ **Custom Exception :-**

- Python allows user to create own exceptions using classes.
- Custom exceptions useful when :
Built – in exceptions are not enough.
We want to meaningful error messages.

Syntax :-

```
Class MyError(Exception):
```

```
    Pass
```

Example :-

```
class NegativeNumberError(Exception):
```

```
    pass
```

```
try:
```

```
    num = int(input("Enter a positive number: "))
```

```
if num < 0:
```

```
    raise NegativeNumberError
```

```
print("You entered:", num)
```

```
except NegativeNumberError:
```

```
print("Negative numbers are not allowed!")
```

12. Understanding the concepts of classes, objects, attributes, and methods in Python.

⇒ **Class** :- class is a collection of variable and functions.

it defines attributes and methods

Attributes :- data/variable

Methods :- functions / behaviors.

Example :-

Class car:

Pass

⇒ **Objects** :- object is an instance of the class.

Example :-

Class car:

Pass

Car1 = car()

Car2 = car()

⇒ **Attributes** :- Attributes are variable that belongs to an object or class.

Example :-

Class person:

```
def __init__(self,name,age,contact):
```

```
    self.name = name
```

```
    self.age = age
```

```
    self.contact = contact
```

Here,

Name , age, contact is my attributes

⇒ **Methods** :-

Methods are functions defined inside the class.

They describe what objects can do.

Example :-

Class car:

```
def __init__(self, name, brand):  
    self.name = name  
    self.brand = brand
```

```
def test(self):  
    print(self.name, self.brand)
```

```
t = test("Grand_Vitara", "Suzuki")  
t.test()
```

13. Difference between local and global variables.

⇒ Below the basic difference between local and global variables.

Property	Local Variable	Global Variable
Scope	Within a function or block where declared	Entire Program
Visibility	Not visible outside the function or block	Visible in all function
Lifetime	Created when function is called, destroyed when it ends	Created at program start, when destroyed when it ends.
Storage Class	Usually Auto(by default)	Static or extern(default is global)

14. Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python.

⇒ There are below the inheritance in python

✓ **Single Inheritance** :- a child class inherits from one parent class

Example :-

```
class parent:  
    def show(self):
```

```
print("Parent class calling")
```

```
class child(parent):  
    def display(self):  
        print("Child class calling")
```

```
c1 = child()  
c1.show()  
c1.display()
```

output :-

Parent class calling
Child class calling

- ✓ **Multilevel Inheritance :-** a class inherits from another derived class.

Example :-

```
class Grandparent:  
    def test(self):  
        print("GrandParent class calling")
```

```
class parent(Grandparent):  
    def show(self):  
        print("Parent class calling")
```

```
class child(parent):  
    def display(self):  
        print("Child class calling")
```

```
c1 = child()  
c1.test()  
c1.show()  
c1.display()
```

output :-

Grandparent class calling
Parent class calling
Child class calling

- ✓ **Multiple Inheritance** :- a child class can inherit from more than one parent class.

Example :-

```
class A:
    def test(self):
        self.num1 = int(input("Enter Number 1 = "))

class B:
    def test1(self):
        self.num2 = int(input("Enter Number 2 = "))

class C(A,B):
    def test2(self):
        self.num3 = int(input("Enter Number 3 = "))

    def add(self):
        print("Addition of two number is = ",self.num1 + self.num2 + self.num3)

c1 = C()
c1.test()
c1.test1()
c1.test2()
c1.add()
```

Output :-

```
Enter Number 1 = 10
Enter Number 2 = 20
Enter Number 3 = 30
Addition of two number is = 60
```

- ✓ **Hierarchical Inheritance** :- multiple child classes can inherit from the one parent class.

Example :-

```
class Vehicle:
    def __init__(self,name,price,brand):
        self.name = name
        self.price = price
        self.brand = brand
```

```

def display(self):
    print(self.name,self.price,self.brand)

class Car(Vehicle):
    def __init__(self,name,price,brand):
        super().__init__(name,price,brand)

class Bike(Vehicle):
    def __init__(self,name,price,brand):
        super().__init__(name,price,brand)

b1 = Bike("Activa",40000,"Honda")
c1 = Car("Maruti800",80000,"Suzuki")

b1.display()
c1.display()

```

Output :-

Activa 40000 Honda
Maruti800 80000 Suzuki

- ✓ **Hybrid Inheritance** :- combination of two or more types of inheritance.

Example :-

```

class vehicle:
    def __init__(self,name,price,color):
        self.name = name
        self.price = price
        self.color = color

    def display(self):
        print(self.name,self.price,self.color)

class car(vehicle):
    def __init__(self,name,price,color):
        super().__init__(name,price,color)

class bike(vehicle):

```

```

def __init__(self,name,price,color):
    super().__init__(name,price,color)

class cycle(car,bike):
    def __init__(self,name,price,color):
        super().__init__(name,price,color)

c1 = car("Bugati",40000,"Red")
b1 = bike("BMW",60000,"GTR")
c2 = cycle("Downhill",70000,"Zipper")

c1.display()
b1.display()
c2.display()

```

Output :-

```

Bugati 40000 Red
BMW 60000 GTR
Downhill 70000 Zipper

```

15. Method overloading: defining multiple methods with the same name but different parameters.

⇒ In python, multiple methods can have same name but different parameter that is known as Method overloading

Unlike languages such as Java and C++, but python does not support the method overloading directly.

Achieves the method overloading in a different way using multipledispatch method

Example :-

```

from multipledispatch import dispatch

```

```

class calc:
    @dispatch(int,int)
    def add(self,a,b):

```

```

        print(f"Addition of {a} and {b} is {a+b}")
    @dispatch(int,int,int)

    def add(self,a,b,c):
        print(f"Addition of {a}, {b} and {c} is {a+b+c}")

c = calc()
c.add(10,20)
c.add(10,20,30)

```

Output :-

Addition of 10 and 20 is 30

Addition of 10, 20 and 30 is 60

16. Method overriding: redefining a parent class method in the child class.

⇒ Method overriding means redefining a method of the parent class to child class with the same method name and parameter that is known as Method Overriding.

The child class provides implementation of the parent class method

Example :-

```

class A:
    def test(self):
        print("Calling A")

```

```

class B:
    def test(self):
        print("Calling B")

```

```

b = B()
b.test()

```

Output :-

Calling B

17. Using re.search() and re.match() functions in Python's re module for pattern matching.

⇒ Below the method :-

✓ **Re.match() :-**

Checks for a match only at the beginning of the string.

Syntax :- `re.match(pattern, string)`

Example :-

```
Import re
text = "python is easy"
k = re.match("python",text)
if k:
    print("match found")
else:
    print("match not found")
```

Output :-

Match found

✓ **Re.search() :-**

Searches for a pattern anywhere in the string

Syntax :- `re.search(pattern, string)`

Example :-

```
Import re
text = "python is easy"
k = re.search("on",text)
if k:
    print("match found")
else:
    print("match not found")
```

Output :-

Match found

18. Difference between search and match.

⇒ Below the difference :-

Re.match()	Re.search()
Matches pattern only at the beginning of the string	Searches pattern anywhere in the string
Start of the string only	Entire string
Pattern is found at position 0	Pattern is found anywhere
Slightly Faster	Slightly slower
Validating starting format	Finding pattern inside text

19. Introduction to SQLite3 and PyMySQL for database connectivity.

⇒ Below the explanation of SQLite3 and PyMySQL

✓ **SQLite3 :-**

- It is a lightweight , file based database system included with python.
- No installation is required
- Stores data in a single file
- Suitable for small application

Example :-

```
Import sqlite3
```

```
Con = sqlite3.connect('test.db')
```

```
Cursor = con.cursor()
```

```
Cursor.execute('create table student(id int primary key, name varchar(50), age int)')
```

```
Con.commit()
```

```
Con.close()
```

✓ **PyMySQL :-**

- It is a python library used to connect a MySQL database server.
- Works with MySQL database.
- Suitable for web and large scale application.

- Requires Installation

Example :-

Installation :- pip install pymysql

```
Import mysql
```

```
Con = pymysql.connect(  
    Host = "localhost",  
    User = "root",  
    Password = "root",  
    Database = "college"  
)
```

```
Cursor = con.cursor()
```

```
Print("Connected successfully!...")
```

```
Con.close()
```